DAY 05 | PRE-READ

The Web's Postal Service

A light read before Day 5. About 15 minutes, no code required.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Why read this?

Yesterday you saw that clients talk to servers through APIs. Tomorrow you will learn the language they speak, called HTTP, the rules that make an API pleasant to use, called REST, and you will build your first API with a tool called FastAPI.

1. HTTP is like sending a letter

Letter	HTTP request	Example
Address on the envelope	URL	shop.com/products/42
What you want done	Method	Please send me (GET), please add (POST)
Notes on the envelope	Headers	Language, who you are
The letter inside	Body	Details of a new order
The reply letter	Response	Status code plus the information

2. Five common requests

- **GET**: show me something. Like looking at a menu.
- **POST**: create something new. Like placing an order.
- **PUT**: replace something completely. Like rewriting your address.
- **PATCH**: change one detail. Like fixing a typo in your address.
- **DELETE**: remove something. Like cancelling an order.

3. The reply always starts with a number

Every response comes with a three digit **status code**, a quick way of saying how things went. You already know one: **404 Not Found**, the page that does not exist.

Starts with	Meaning	Everyday version
2	Success	Your parcel was delivered.
4	Your mistake	Wrong address on the parcel.
5	Their mistake	The post office lost it.

4. REST: good manners for APIs

REST is a set of simple habits. Name things with nouns (products, orders). Use the request type to say what to do. Do not expect the server to remember you between requests; send what it needs every time. When everyone follows the same manners, any developer can use any API quickly.

5. About tomorrow's assessment

Tomorrow includes a 2 hour assessment on Days 1 to 4. Revise your post-reads and workbooks tonight, and re-run the notebooks. Focus on understanding, not memorising.

6. Think about these before class

- When you search on a shopping site, which request type do you think your browser sends?
- What status code might you get if you try to open someone else's private account page?
- Why might it be useful for a server not to remember previous requests?

Ganbatte kudasai (頑張ってください): please do your best, for the assessment